

GREENLIGHT

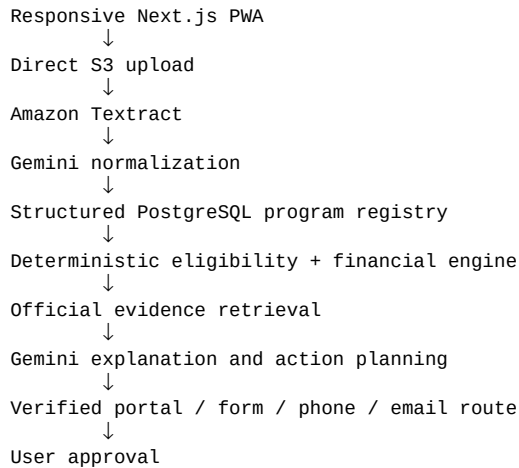
Technical Architecture, Product Requirements & Hackathon Build Plan

AWS + Textract + Gemini + PostgreSQL + Evidence-First Eligibility + Action Routing

Technical synthesis of the Greenlight build brief and validated architecture decisions.

1. Executive Technical Verdict

Greenlight should use a hybrid, evidence-first architecture. Document extraction should be deterministic and traceable; AI should normalize and explain; eligibility and financial calculations should be performed by application code; and action routing should use verified official destinations.



Core principle: AI interprets. Code calculates. Evidence supports the claim. The user approves consequential actions.

2. Key Architecture Corrections

2.1 Do not use Gemini as the only OCR layer

- Amazon Textract should provide text, coordinates, confidence and document structure.
- Gemini should semantically normalize extracted information into the application's canonical bill schema.
- Zod or equivalent schema validation should reject malformed model output.
- Conflicting or low-confidence values should become user-confirmation tasks.

2.2 RAG must not decide eligibility

- RAG retrieves authoritative passages and evidence.
- Structured program rules determine eligibility.
- Benefit amounts should come from official tables or deterministic formulas.
- Every recommendation should retain the program version and source snapshot that produced it.

2.3 Action routing must be route-aware

- Do not assume every program accepts email.
- Resolve the official route first: portal, web form, phone, mail, intake agency, utility election form or manual review.
- Gemini may draft content only after the backend supplies a verified route.
- Never invent departments, email addresses or administrators.

2.4 Separate savings from financing

- Monthly bill credits, rebates, grants and estimated operating savings are different financial categories.

- Financing should never be counted as savings.
- Upfront costs should be represented separately from benefits.
- Use a value ledger instead of one opaque total.

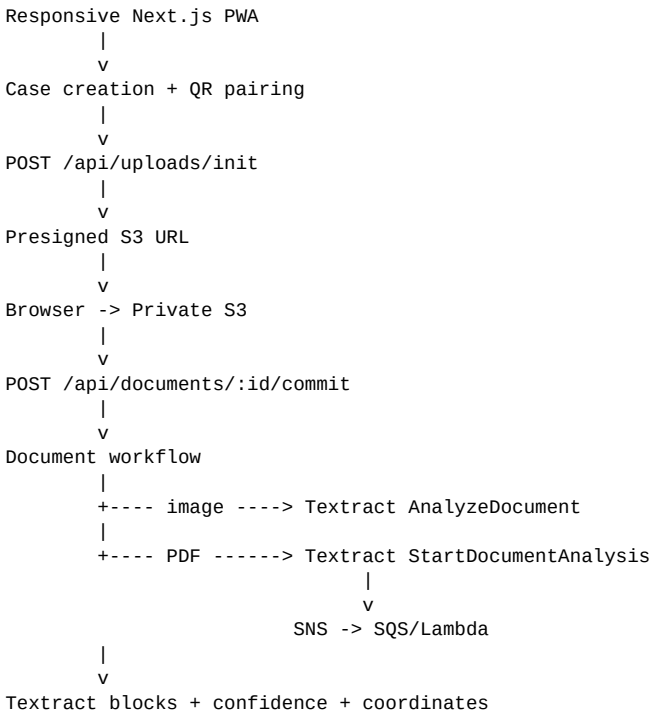
2.5 Avoid fake confidence scores

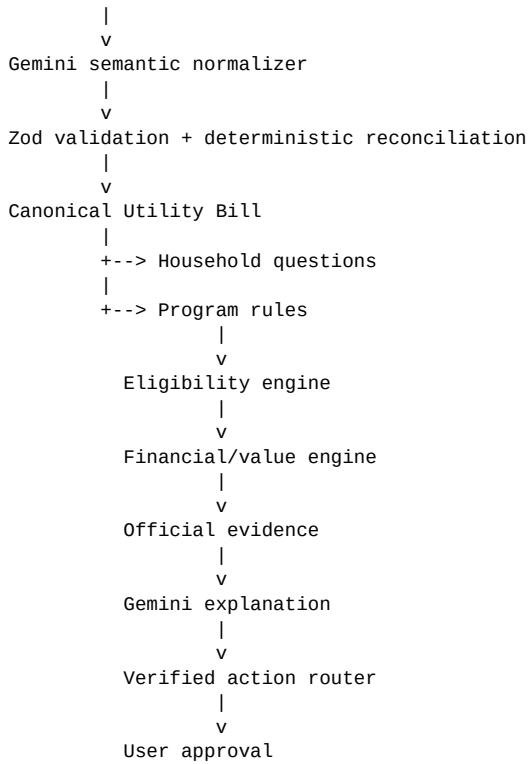
- Prefer confirmed requirements, missing requirements and program status.
- Use statuses such as eligible, likely eligible, possible match, ineligible and manual review.
- Expose evidence coverage rather than pretending an LLM confidence value is an eligibility probability.

3. Technology Comparison

Layer	Recommended	Alternative	Decision
Frontend	Next.js PWA	Expo	Next.js for hackathon
OCR	Amazon Textract	Google Document AI / Azure DI	Textract primary
AI	Google GenAI SDK	Bedrock models	Gemini for reasoning
Database	PostgreSQL	DynamoDB	Postgres for rules
Hackathon DB	Supabase	Aurora	Supabase fastest
Production DB	Aurora PostgreSQL	Supabase	Aurora AWS-native
Evidence RAG	Bedrock KB	Custom pgvector	Optional P1
Storage	Private S3	Supabase Storage	S3 for AWS pipeline
Workflow	Lambda/SQS/SNS	Step Functions	Simple event flow first

4. End-to-End Architecture





5. Document Processing

5.1 Upload

```
POST /api/cases
POST /api/uploads/init
POST /api/documents/{documentId}/commit
GET  /api/cases/{caseId}/status
GET  /api/cases/{caseId}/result
```

- Use direct browser-to-S3 upload with a short-lived presigned URL.
- Do not proxy large files through the application API.
- Validate MIME type, extension, size and file signature.
- Use random object keys and private bucket access.

5.2 Textract queries

```
What is the utility provider?
What is the total amount due?
What is the due date?
What is the billing period start date?
What is the billing period end date?
How many billing days are included?
What is the total electricity usage in kWh?
What electricity price plan is used?
What is the on-peak usage?
What is the mid-peak usage?
What is the off-peak usage?
What is the service postal code?
What is the account number?
```

5.3 Processing strategy

- Images: synchronous AnalyzeDocument path.

- Multi-page PDFs: asynchronous StartDocumentAnalysis path.
- SNS/SQS/Lambda can handle completion events.
- For the live hackathon demo, use a clean single-page synthetic bill.
- Keep a fixture extractor so a cloud timeout cannot break the presentation.

6. Canonical Data Contracts

6.1 Extracted field

```
ExtractedField<T> {
  value: T | null
  rawValue: string | null
  source: textract_query | textract_form | textract_line | gemini | user
  confidence: number | null
  evidence: BoundingBox[]
  needsConfirmation: boolean
  validationErrors: string[]
}
```

6.2 Utility bill

```
UtilityBill {
  documentId
  documentType
  provider
  accountNumberMasked
  billingPeriodStart
  billingPeriodEnd
  billingDays
  totalDueCad
  currentChargesCad
  arrearsCad
  electricityUsageKwh
  naturalGasUsageM3
  ratePlan
  timeOfUse
  serviceLocation
  extractionVersion
}
```

6.3 Program rule

```
EligibilityRule {
  id
  field
  operator
  expected
  required
  hardFailure
  sourceId
  sourceVersionId
}
```

7. Database Design

```
programs
program_versions
program_sources
jurisdictions
administrators
action_routes
eligibility_rules
benefit_rules
required_documents
documents
```

```
extracted_fields
households
cases
case_answers
case_program_evaluations
value_components
evidence_items
agent_events
application_packets
communications
audit_events
```

Program versions are essential because government eligibility thresholds, benefit amounts and application routes can change. Never overwrite a rule without retaining the source version.

8. Eligibility Engine

```
RuleResult =
    PASS | FAIL | UNKNOWN | MANUAL_REVIEW

if required rule FAIL:
    status = ineligible

else if any MANUAL_REVIEW:
    status = manual_review

else if all required rules PASS:
    status = eligible

else if only a small number of required fields are UNKNOWN:
    status = likely_eligible

else:
    status = possible_match
```

The UI should communicate exactly what is known and what is missing. Example: “3 of 5 required conditions confirmed; 2 answers still needed.”

9. Financial Engine

9.1 Normalize usage

```
dailyUsageKwh = totalUsageKwh / billingDays
annualizedUsageKwh = dailyUsageKwh * 365

normalizedChangePercent =
    ((currentKwh / currentDays) -
     (previousKwh / previousDays))
    / (previousKwh / previousDays) * 100
```

9.2 Separate value categories

```
ValueKind =
    monthly_bill_credit
    one_time_rebate
    emergency_grant
    estimated_annual_savings
    no_cost_upgrade
    financing
    upfront_cost
```

9.3 Retrofit economics

```
netUpfrontCost =
    max(0, projectCost - verifiedRebates)

simplePaybackYears =
```

```
annualEstimatedSavings > 0
? netUpfrontCost / annualEstimatedSavings
: null
```

Never claim that a utility bill alone proves which appliance caused a change. Bill-level evidence supports usage and billing patterns, not exact device-level causation.

10. Program Adapter Model

Ontario Electricity Support Program (OESP)

Inputs: household size, combined after-tax income and qualifying account/customer conditions. Output: structured monthly credit and official application route.

Energy Affordability Program (EAP)

Inputs: residency, ownership/tenancy, account holder status, household size, income or qualifying assistance program, heating type and postal prefix. Output: potential no-cost energy-efficiency support and verified delivery-agent route.

Home Renovation Savings Program (HRSP)

Treat rebates as conditional. Validate property, equipment, installation timing and program-specific requirements before presenting an amount as actionable.

Toronto Home Energy Loan Program (HELP)

Treat the amount as financing, not savings. Validate Toronto property and homeowner requirements and route through the official application process.

Low-Income Energy Assistance Program (LEAP)

Use arrears and disconnection-risk conditions. Route through the verified intake-agency process rather than inventing a generic government email.

11. Gemini Responsibilities

- Normalize OCR into strict JSON.
- Explain deterministic rule results using retrieved evidence.
- Select the minimum missing question when multiple programs are candidates.
- Generate route-specific drafts only after the backend resolves an official route.
- Write concise, human-facing explanations.
- Never override deterministic financial values or eligibility rules.

12. Action Router

```
Program candidate
↓
Load current program version
↓
Evaluate structured rules
↓
Any required failure? ---- yes --> Ineligible
|
```

```

no
↓
Missing information? ----- yes --> Ask highest-value question
|
no
↓
Calculate benefit/cost
↓
Retrieve official evidence
↓
Resolve verified administrator/route
↓
Portal / Form / Email / Phone / Agency / Mail
↓
User review
↓
Approval
↓
Execute allowed action or transfer to official destination

```

13. Security & Privacy

- Use synthetic or redacted bills for the hackathon.
- Keep S3 private.
- Use short-lived presigned URLs.
- Never expose AWS, Gemini or database admin credentials to the browser.
- Mask account numbers before sending data to a model.
- Do not log raw OCR text or full bills.
- Delete demo uploads automatically.
- Record audit events without storing unnecessary sensitive content.
- For production, use an approved enterprise/paid AI configuration with appropriate data terms.

14. Recommended Repository Structure

```

greenlight/
├── app/
│   ├── scan/[caseId]/page.tsx
│   ├── analyze/[caseId]/page.tsx
│   ├── results/[caseId]/page.tsx
│   ├── opportunity/[opportunityId]/page.tsx
│   ├── action/[caseId]/[programId]/page.tsx
│   ├── present/[caseId]/page.tsx
│   └── api/{cases, uploads, documents, opportunities, actions}
├── components/
│   ├── BillUploader.tsx
│   ├── DocumentEvidenceViewer.tsx
│   ├── AnalysisProgress.tsx
│   ├── ValueLedger.tsx
│   ├── OpportunityCard.tsx
│   ├── EligibilityMatrix.tsx
│   ├── EvidenceDrawer.tsx
│   ├── ActionRouteCard.tsx
│   └── AgentTimeline.tsx
├── lib/
│   ├── aws/{s3, textract, workflow}.ts
│   ├── ai/{gemini-client, extraction-schema, prompts}.ts
│   ├── extraction/{extractor, textract-image, textract-pdf, gemini-fallback, demo-fixture}.ts
│   ├── programs/{registry, oesp, eap, leap, hrsp, help}.ts
│   ├── eligibility/{evaluator, operators}.ts
│   ├── economics/{bill-normalization, rate-plans, value-ledger, optimizer}.ts
│   └── routing/{resolver, email, portal, phone, agency}.ts

```



```
■ ■■■ security/{pii,file-validation,audit}.ts
■■■ db/
■ ■■■ migrations/
■ ■■■ seeds/
■■■ tests/
    ■■■ eligibility/
    ■■■ economics/
    ■■■ extraction/
    ■■■ routing/
```

15. Environment Variables

```
APP_URL=https://greenlight.example.com

AWS_REGION=ca-central-1
AWS_S3_BUCKET=greenlight-private-documents
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=

GEMINI_API_KEY=
GEMINI_FAST_MODEL=
GEMINI_REASONING_MODEL=

DATABASE_URL=

NEXT_PUBLIC_DEMO_MODE=true
DEMO_DOCUMENT_TTL_HOURS=24
PRESIGNED_UPLOAD_TTL_SECONDS=600

ENABLE_EXTRACT=true
ENABLE_GEMINI_FALLBACK=true
ENABLE_BEDROCK_EVIDENCE=false
```

16. Six-Hour Hackathon Execution

Time	Frontend / UX	Document / AWS	Rules / AI
0–1h	Scaffold + case + QR	S3 + presigned upload	Seed programs
1–2h	Upload + progress + results	Textract extraction	Eligibility engine
2–3h	Evidence + opportunity cards	Gemini normalization	Financial/value engine
3–4h	Get-it-for-me flow	Fixture fallback + status	Evidence + action route
4–5h	Mobile/stage polish	Integration	One complete program action
5–6h	Rehearse + reset demo	Failure recovery	Final verification

17. Do Not Build During the Six Hours

- Native Expo application.
- Full government web crawler.
- General-purpose autonomous agent framework.
- Full Bedrock Knowledge Base.
- Gmail/Outlook OAuth.
- Automatic external message sending.
- Multi-province support.

- Full demographic modeling.
- Contractor marketplace.
- Advanced multi-page PDF support on the critical demo path.
- Chatbot tab or generic conversational interface.
- Admin dashboard.

18. Demo Resilience

```

ExecutionMode =
  live | hybrid | demo

LIVE:
  real S3 + real Textract + real Gemini + real rules

HYBRID:
  real upload + real Textract + cached AI narrative + real rules

DEMO:
  synthetic bill + saved extraction + saved program evidence
  + real deterministic rules + saved narrative

```

The UI and calculation engine should remain the same in all modes. Only the external dependency adapters change.

19. Definition of Done

- Laptop creates a case and displays a QR code.
- Phone opens the case and captures/uploads a bill.
- Bill uploads directly to private S3.
- Textract extracts core bill fields.
- Evidence can be traced to the document.
- Low-confidence fields can be confirmed.
- At least four Ontario programs are seeded.
- Eligibility uses deterministic rules.
- Official evidence is attached to recommendations.
- Savings, rebates, grants and financing are separated.
- One high-value missing question can be asked.
- One official action route can be resolved.
- No contact information is invented.
- User reviews consequential actions.
- Mobile and desktop views work.
- Demo fallback works.
- The story completes in under three minutes.

20. Final Technical Position

Greenlight should not present itself as an AI bill scanner. Its strongest technical story is an evidence pipeline: AWS reads the document, deterministic software calculates the financial and eligibility result, Gemini explains the bureaucracy, and a verified action router guides the user to the correct next step.

The most important engineering boundary is simple: models can interpret and explain, but they should not be the authority for financial arithmetic, program thresholds, or government contact destinations.